

## MINI-RAPPORT TECHNIQUE

### API Service — Médiathèque Numérique

*Backend REST Spring Boot pour la gestion d'une médiathèque (livres, utilisateurs, emprunts)*

Dépôt : [github.com/MahefasoaRAM/java-project](https://github.com/MahefasoaRAM/java-project) · Date : 30 juillet 2026

## 1. Introduction

Ce mini-rapport présente une analyse technique du projet **java-project**, une API REST développée en Java avec le framework Spring Boot, destinée à la gestion d'une médiathèque numérique. L'objectif du système est de digitaliser trois processus métier centraux : la gestion du catalogue de livres, la gestion des comptes utilisateurs, et le suivi des emprunts, tout en garantissant un contrôle d'accès strict par rôle.

Le projet est conçu comme un **service backend autonome**, exposant une API JSON consommable par une application cliente (web ou mobile) qui ne fait pas partie du périmètre de ce dépôt. Il répond à des besoins opérationnels concrets : traçabilité des emprunts, disponibilité en temps réel des ouvrages, et sécurisation des opérations sensibles (ajout/suppression de livres) via un mécanisme d'authentification.

## 2. Architecture technique

Le projet suit une **architecture en couches** (layered architecture), un modèle éprouvé qui sépare clairement les responsabilités et facilite la maintenance et les tests :

| Couche          | Responsabilité                                          |
|-----------------|---------------------------------------------------------|
| controller      | Expose les endpoints HTTP et gère les requêtes/réponses |
| service         | Contient la logique métier et les règles de gestion     |
| repository      | Accès aux données via Spring Data JPA                   |
| entity          | Modèle persistant (mappé en base de données)            |
| dto / request   | Objets d'échange en entrée/sortie de l'API              |
| config / filter | Sécurité JWT et configuration Spring Security           |

### Choix technologiques et intérêt

| Technologie           | Intérêt dans le contexte du projet                                                                                                                                  |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Java 17               | Version LTS (Long-Term Support) : stabilité, performances et support à long terme, avec des fonctionnalités modernes du langage (records, pattern matching).        |
| Spring Boot 3.4.2     | Réduit fortement la configuration nécessaire pour démarrer une API REST ; fournit un écosystème mature (sécurité, persistance, documentation) parfaitement intégré. |
| Spring Security + JWT | Permet une authentification sans état (stateless), idéale pour une API REST consommée par plusieurs types de clients, sans gestion de session serveur.              |
| Spring Data JPA       | Abstrait l'accès aux données et réduit le code répétitif (requêtes dérivées automatiquement des noms de méthode).                                                   |
| MySQL                 | Système de gestion de base de données relationnelle robuste, adapté à un modèle de données avec relations claires (utilisateur-emprunt-livre).                      |

| Technologie                    | Intérêt dans le contexte du projet                                                                                                              |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Lombok                         | Élimine le code répétitif (getters/setters/constructeurs), améliore la lisibilité des entités et DTO.                                           |
| Springdoc OpenAPI / Swagger UI | Génère automatiquement une documentation interactive de l'API, directement à partir du code, facilitant les tests et l'intégration côté client. |

### 3. Fonctionnalités principales

#### Authentification

- Inscription d'un nouvel utilisateur ;
- Connexion par email et mot de passe ;
- Génération d'un token JWT à l'inscription et à la connexion ;
- Protection des routes sensibles via Spring Security.

#### Gestion des livres

- Ajout, modification et suppression d'un livre (réservés au rôle ADMIN) ;
- Consultation de la liste complète du catalogue ;
- Recherche ciblée par identifiant, titre, auteur ou catégorie.

#### Gestion des utilisateurs

- Création, modification et suppression d'un compte utilisateur ;
- Consultation de la liste des utilisateurs ;
- Filtrage par rôle ou par email.

#### Gestion des emprunts

- Création d'un emprunt pour un livre disponible ;
- Retour d'un livre emprunté ;
- Consultation de l'historique complet des emprunts ;
- Consultation des emprunts actuellement en cours.

### 4. Modèle de données

À partir des fonctionnalités exposées, trois entités principales structurent le domaine métier :

| Entité      | Attributs principaux (déduits)                                         | Rôle                                                                      |
|-------------|------------------------------------------------------------------------|---------------------------------------------------------------------------|
| Utilisateur | id, nom, email, mot de passe, rôle (ADMIN / USER)                      | Représente un compte capable de s'authentifier et d'emprunter des livres. |
| Livre       | id, titre, auteur, catégorie, disponibilité                            | Représente un ouvrage du catalogue, avec un statut de disponibilité.      |
| Emprunt     | id, date d'emprunt, date de retour prévue, statut (en cours / terminé) | Entité de liaison qui associe un Utilisateur à un Livre sur une           |

| Entité | Attributs principaux (déduits) | Rôle            |
|--------|--------------------------------|-----------------|
|        |                                | période donnée. |

### Relations entre entités

- **Utilisateur** → **Emprunt** (relation 1 – N) : un utilisateur peut posséder plusieurs emprunts, mais chaque emprunt appartient à un seul utilisateur.
- **Livre** → **Emprunt** (relation 1 – N) : un livre peut faire l'objet de plusieurs emprunts au fil du temps, mais ne peut avoir qu'un seul emprunt actif à un instant donné.
- L'entité **Emprunt** constitue ainsi une table d'association enrichie (avec ses propres attributs de date et de statut) entre **Utilisateur** et **Livre**, modélisant une relation N–N indirecte via une entité intermédiaire — un choix de modélisation classique et approprié pour tracer l'historique des emprunts.

## 5. Sécurité et authentification

La sécurité repose sur un mécanisme **JWT (JSON Web Token)** combiné à Spring Security, avec une gestion d'accès par rôle :

| Aspect               | Détail                                                                                           |
|----------------------|--------------------------------------------------------------------------------------------------|
| Endpoints publics    | POST /api/auth/register et POST /api/auth/login                                                  |
| Routes protégées     | Toutes les autres routes nécessitent un token JWT valide (en-tête Authorization: Bearer <token>) |
| Contrôle par rôle    | Les opérations de création, modification et suppression de livres sont réservées au rôle ADMIN   |
| Nature de la session | Stateless (sans session serveur), adaptée à une architecture REST                                |

Ce choix d'architecture **stateless** présente un intérêt majeur pour la scalabilité : le serveur n'a pas besoin de conserver l'état de connexion de chaque client, ce qui simplifie la mise à l'échelle horizontale (plusieurs instances de l'API peuvent traiter indifféremment les requêtes, sans partage de session).

## 6. Points d'entrée de l'API (endpoints clés)

### Authentification

| Méthode | Chemin             | Description                          |
|---------|--------------------|--------------------------------------|
| POST    | /api/auth/register | Inscription d'un nouvel utilisateur  |
| POST    | /api/auth/login    | Connexion et génération du token JWT |

### Livres

| Méthode | Chemin      | Description              |
|---------|-------------|--------------------------|
| POST    | /api/livres | Ajouter un livre (ADMIN) |
| GET     | /api/livres | Lister tous les livres   |

| Méthode | Chemin                            | Description                        |
|---------|-----------------------------------|------------------------------------|
| GET     | /api/livres/{id}                  | Consulter un livre par identifiant |
| GET     | /api/livres/titre/{titre}         | Consulter un livre par titre       |
| GET     | /api/livres/auteur/{auteur}       | Consulter les livres par auteur    |
| GET     | /api/livres/categorie/{categorie} | Consulter les livres par catégorie |
| PUT     | /api/livres/{id}                  | Modifier un livre (ADMIN)          |
| DELETE  | /api/livres/{id}                  | Supprimer un livre (ADMIN)         |

## Utilisateurs

| Méthode | Chemin                     | Description                              |
|---------|----------------------------|------------------------------------------|
| POST    | /api/users                 | Créer un utilisateur                     |
| GET     | /api/users                 | Lister tous les utilisateurs             |
| GET     | /api/users/{id}            | Consulter un utilisateur par identifiant |
| GET     | /api/users/email/{email}   | Consulter un utilisateur par email       |
| GET     | /api/users/role/{roleUser} | Filtrer les utilisateurs par rôle        |
| PUT     | /api/users/{id}            | Modifier un utilisateur                  |
| DELETE  | /api/users/{id}            | Supprimer un utilisateur                 |

## Emprunts

| Méthode | Chemin                    | Description                               |
|---------|---------------------------|-------------------------------------------|
| POST    | /api/emprunts             | Créer un emprunt pour un livre disponible |
| PUT     | /api/emprunts/{id}/retour | Retourner un livre emprunté               |
| GET     | /api/emprunts             | Lister tous les emprunts                  |
| GET     | /api/emprunts/en-cours    | Lister les emprunts en cours              |

## 7. Règles métier

- Un livre ne peut être emprunté que s'il est disponible ;
- Un livre ne peut avoir qu'un seul emprunt actif à la fois ;
- Un utilisateur peut avoir plusieurs emprunts simultanés ;
- Le retour d'un livre rend automatiquement celui-ci de nouveau disponible ;
- Un livre ne peut pas être supprimé s'il fait encore l'objet d'un emprunt en cours.

Ces règles, appliquées au niveau de la couche **service**, garantissent la cohérence des données indépendamment du client qui consomme l'API — un principe essentiel pour une API destinée à être partagée entre plusieurs applications front-end.

## 8. Documentation et tests

### Documentation Swagger / OpenAPI

La documentation interactive de l'API est générée automatiquement via **springdoc-openapi** et accessible aux adresses suivantes :

- Interface Swagger UI : <http://localhost:8080/swagger-ui/index.html>
- Endpoint OpenAPI JSON : <http://localhost:8080/v3/api-docs>

Cette documentation permet de visualiser l'ensemble des endpoints, de tester les requêtes directement depuis le navigateur, et de renseigner un token JWT pour authentifier les appels protégés — un gain de temps notable pour l'équipe de développement et pour les intégrateurs front-end.

### Tests

Le projet inclut des **tests unitaires** (JUnit 5 et Mockito) couvrant la couche service, où se concentre l'essentiel de la logique métier (gestion de la disponibilité des livres, statuts d'emprunt, règles de suppression). Cette approche cible les scénarios à plus forte valeur de validation plutôt qu'une couverture exhaustive de toutes les couches.

## 9. Conclusion

---

### Points forts

- Architecture en couches claire, conforme aux standards professionnels Spring Boot ;
- Sécurité JWT stateless avec contrôle d'accès par rôle, adaptée à une API REST ;
- Règles métier cohérentes et centralisées dans la couche service ;
- Documentation Swagger opérationnelle, facilitant l'intégration côté client ;
- Présence de tests unitaires sur la logique métier critique.

### Pistes d'amélioration possibles

- Ajouter des tests d'intégration (par exemple avec Testcontainers) pour valider le comportement de bout en bout, y compris la couche sécurité ;
- Renforcer le contrôle d'accès sur les routes utilisateur, en vérifiant qu'un utilisateur ne peut consulter ou modifier que son propre compte (sauf pour un ADMIN) ;
- Externaliser la configuration sensible (secret JWT, identifiants de base de données) via des variables d'environnement plutôt que des valeurs en dur dans le code versionné ;
- Mettre en place une politique de gestion des tokens (rafraîchissement, révocation) pour renforcer la sécurité sur la durée ;
- Ajouter un système de pagination sur les listes (livres, utilisateurs, emprunts) en prévision d'une montée en charge du catalogue.

En synthèse, ce projet constitue une base backend solide et bien structurée pour une médiathèque numérique, exploitant judicieusement l'écosystème Spring Boot moderne. Les évolutions proposées ci-dessus relèvent principalement du durcissement en vue d'une mise en production, plus que d'une refonte structurelle.